オブジェクト指向言語

第5回

機能のライブラリ化

講義内容（予定）

- 第1回（2026/09/18）　講義概要・開発環境の導入・変数と型
- 第2回（2026/09/25）　Java: 制御構造、配列、メソッド
- 第3回（2026/10/02）　文字列、ファイル入出力と例外
- 第4回（2026/10/09）　代表的なデータ構造(1)　←――【テスト】
- 第5回（2026/10/16）　機能のライブラリ化
- 第6回（2026/10/23）　代表的なデータ構造(2)
- 第7回（2026/11/06）　GUIプログラミング・GUI作成実習1
- 第8回（2026/11/13）　GUI作成実習2　←――【テスト】
- 第9回（2026/11/20）　画面描画、インタフェース、ポリモルフィズム
- 第10回（2026/11/27）　ネットワーク、スレッド
- 第11回（2026/12/04）　プロジェクト(1)　←――【テスト】
- 第12回（2026/12/11）　プロジェクト(2)
- 第13回（2026/12/18）　プロジェクト(3) 中間報告
- 第14回（2027/01/08）　プロジェクト(4) デモ
- 第15回（2027/01/22）　プロジェクト(5) 成果発表

下線の回は出張の予定ですが、教室に集合 + TA/SAサポートで演習実施

レポート4 解説 (1)

```
private void append(int v){
  if(head == null){
    insert(v);
  }else{
    Cell c = head;
    while(c.next != null){
      c = c.next;
    }
    c.next = new Cell();
    c.next.value = v;
    c.next.next = null;
  }
}
```

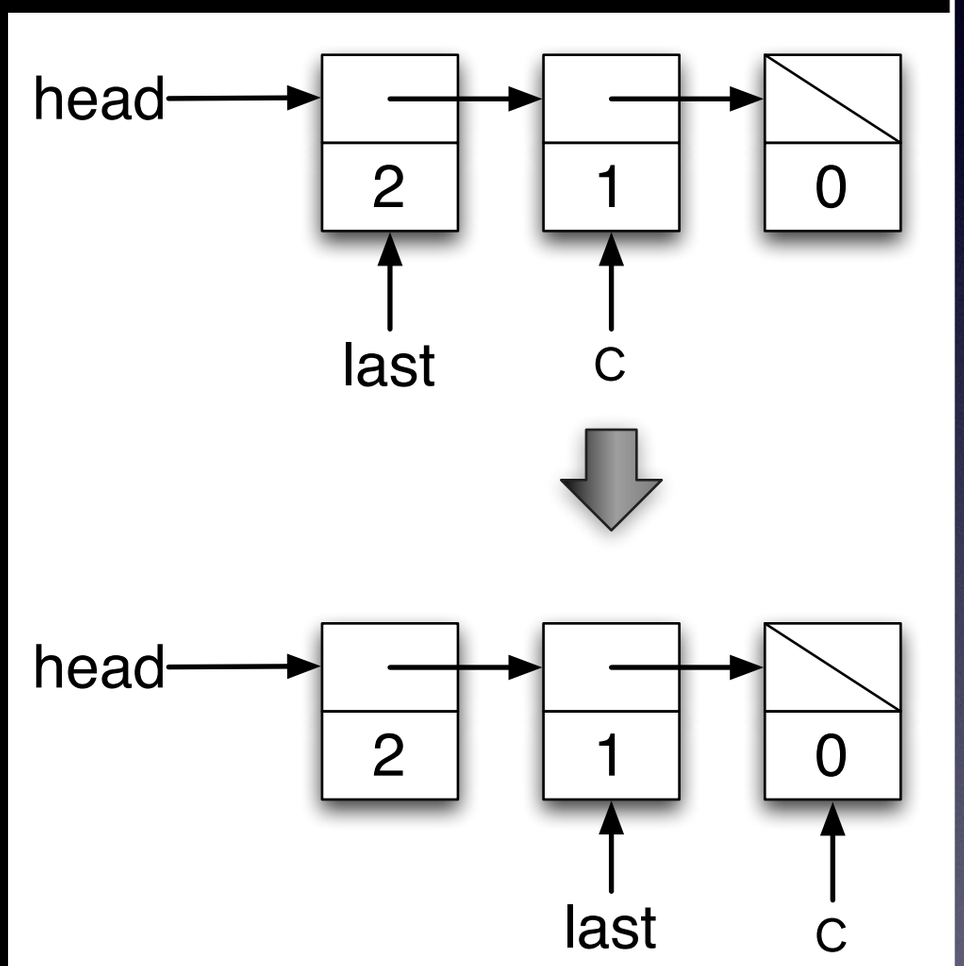
空のリストなら先頭に入れる

空でないなら、末尾までたどってから追加する

レポート4 解説 (2)

```
public void delete(int v){
  Cell c = head;
  Cell last = head;
  while(c != null){
    if(c.value == v){
      if(c == head){
        head = c.next;
      }else{
        last.next = c.next;
      }
    }else{
      last = c;
    }
    c = c.next;
  }
}
```

一致しない場合

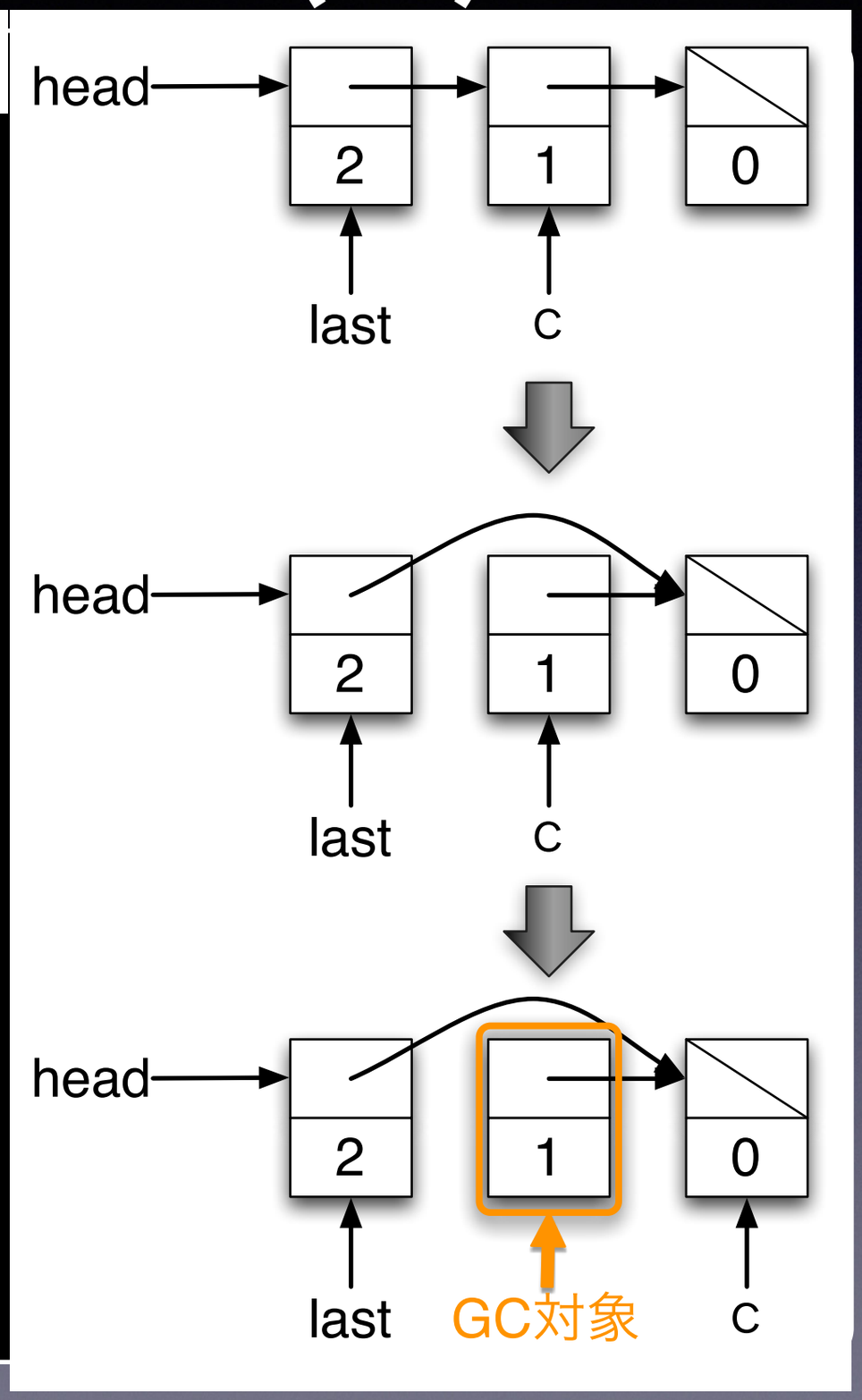


(スペースの都合でセルの絵は縦並びにした)

レポート4 解説 (3)

途中が該当する場合

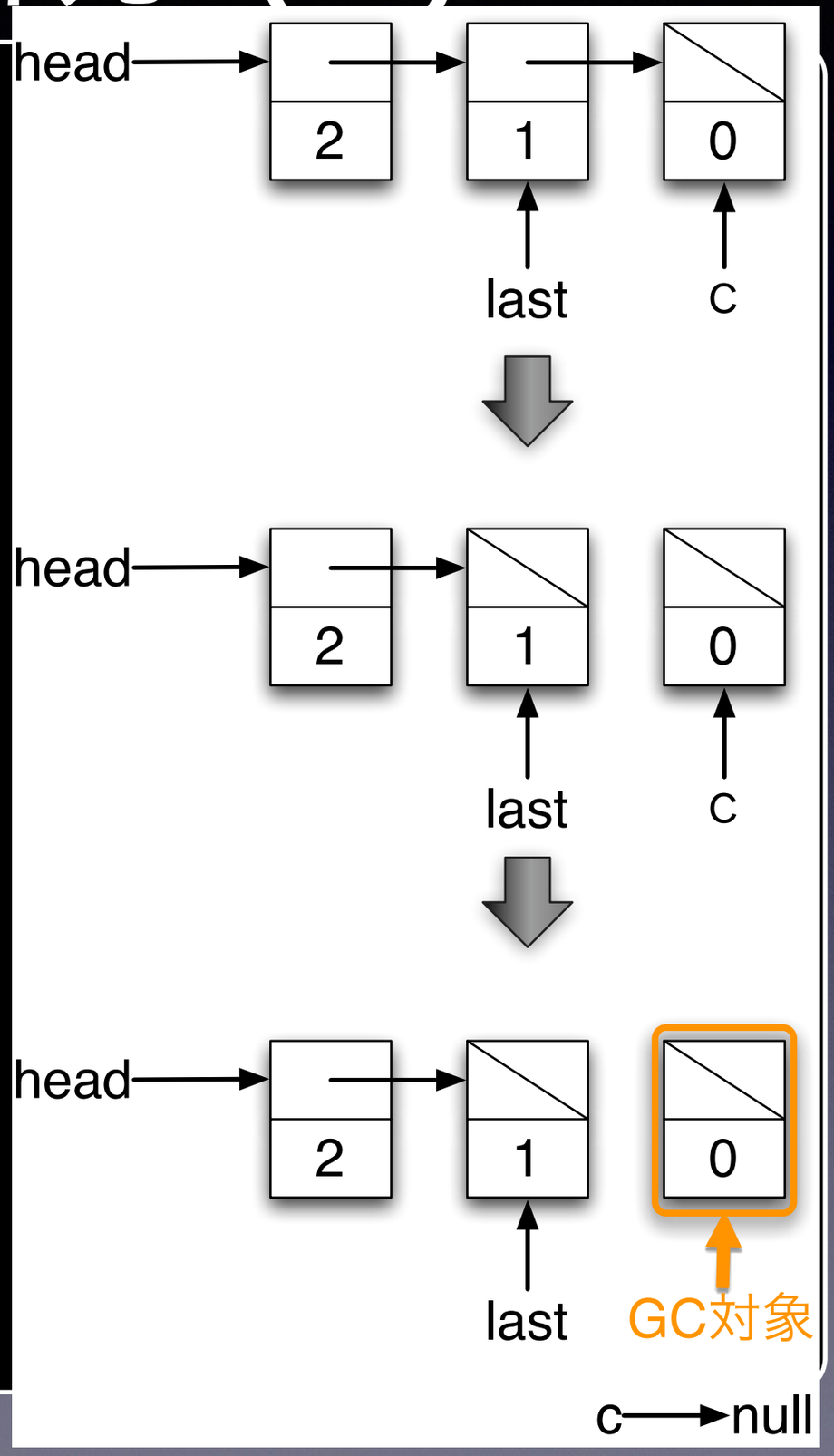
```
public void [redacted]{
  Cell c = head;
  Cell last = head;
  while(c != null){
    if(c.value == v){
      if(c == head){
        head = c.next;
      }else{
        last.next = c.next;
      }
    }else{
      last = c;
    }
    c = c.next;
  }
}
```



レポート4 解説 (4)

末尾が該当する場合

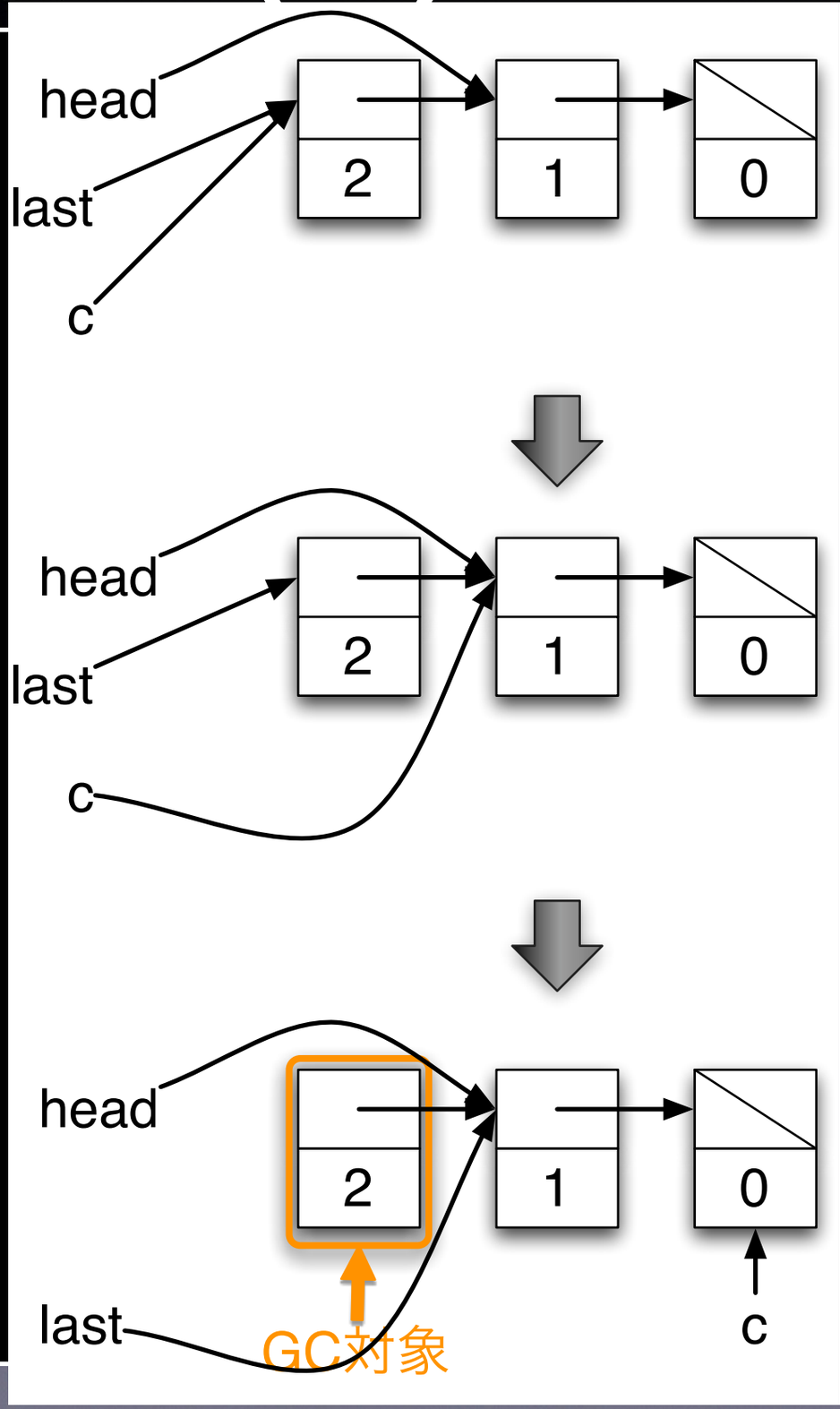
```
public void [redacted]{
  Cell c = head;
  Cell last = head;
  while(c != null){
    if(c.value == v){
      if(c == head){
        head = c.next;
      }else{
        last.next = c.next;
      }
    }else{
      last = c;
    }
    c = c.next;
  }
}
```



レポート4 解説 (5)

先頭が該当する場合

```
public void delete(int v){
  Cell c = head;
  Cell last = head;
  while(c != null){
    if(c.value == v){
      if(c == head){
        head = c.next;
      }else{
        last.next = c.next;
      }
    }else{
      last = c;
    }
    c = c.next;
  }
}
```



リスト操作の
ライブラリ化(1)

- ListSampleの問題点
  - リスト操作 insert(), append(), delete() の定義が具体的な操作（リストへのデータの出し入れ）と混在
  - リストの利用者 ＝ main()側が、Cellを直接操作できる → リストの構造を破壊できる

リスト操作のライブラリ化(2)

- 望ましい形は…
  - 「リストの機能」は別パッケージにする
    - Cellはそのパッケージ内部だけで利用する
  - main()はリスト機能の利用者側なので、そこには含めない ＝ 独立したパッケージにする

リスト操作のライブラリ化(3)

- Eclipseを起動
- プロジェクトを作成 Week05
- パッケージを作成
  - メールアドレス逆順 + ListLib
- 明星MoodleからJavaファイル x 2を入手
- 仮想Windows環境にアップロード→2つのJavaファイルを自分のパッケージにドラッグ＆ドロップ
- 赤×をクリック → Add package declaration … + Save

リスト操作のライブラリ化(4)

- 実行して、動作することを確認せよ
- リストを表示する部分をメソッド化(22-29行目)
  - マウスで選択 → 右クリック → Refactor > Extract method
  - メソッド名は printList() 、Access modifier は public ← 機能提供のため
- 必要に応じて Source > Correct Indentation
- Cell c; の宣言が不要になるので消す
  - 黄! をクリック → Remove 'c' and ...

リスト操作のライブラリ化(5)

- main()側のパッケージを作成
  - パッケージ名は ListLibMain
- ListLibMain クラスを作成
  - ただし main() は作成しない
- ListSampleクラスのmain()とコンストラクタをListLibMainにドラッグ＆ドロップ
- ListSampleクラスをListに名前変更
  - ListSample.javaを右クリック → Refactor > Rename

リスト操作のライブラリ化(6)

- ListへのRenameでエラーが出るはず
  - ListLibMain 側で名前に食い違いが出るので
- Next して ListLibMain.java の修正案を見ると、import 部分が修正されるのが分かる → Finish

リスト操作のライブラリ化(7)

- ListLib側の整備
  - Cellをパッケージ外部に公開しない
    - publicを削除 ＝ 同一パッケージ内だけで利用可能 (package private)
- Listクラスの整備
  - こちらは public のまま
  - headを外部に公開しない ＝ private
  - 3つのメソッドはパッケージ外部から呼べるようpublicに変更

リスト操作のライブラリ化(8)

- ListLib側の整備（つづき）
  - headの初期化をコンストラクタに移動
    - コンストラクタの作成
      - Source > Generate constructors from super class
      - After 'head'
      - Omit call to … にチェック
    - head = null; をその先頭に追加
    - TODOは消す
    - headの宣言部分の代入は削除

リスト操作のライブラリ化(9)

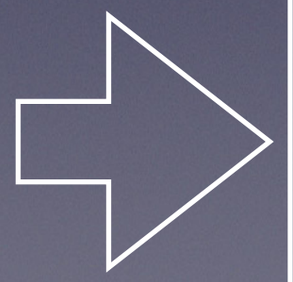
- ListLibMain側の整備
  - もし import … Cell; があったら行ごと削除
    - パッケージ外からは見えないので
  - コンストラクタの赤をクリック → Change to Constructor
  - main()の new List() を new ListLibMain() に変更

リスト操作の
ライブラリ化(10)

- ListLibMain側の整備（つづき）
  - コンストラクタ冒頭 List l = new List();
    - ListLib.Listがimport済みなのを確認
  - メソッド呼び出しに l. を追加 ← 小文字のL

    → エラーが全て解消したことを確認

同一プログラム内の
メソッド呼び出し



Listクラスのオブジェクトが
持つメソッド(機能)の
呼び出し

リスト操作のライブラリ化(11)

- 実行ボタンの右横の三角形をクリック
- Run configurationでMain classを変更
- 実行して結果を確認（当然ながら同じ）

ライブラリ化のまとめ

- リスト機能の提供側と利用側を分離
  - 提供側 (ListLibパッケージ)：
    Cellの定義、リスト操作メソッドの定義
  - 利用側 (ListLibMainパッケージ)：
    リストへのデータ出し入れのみ
- 適切なアクセス制御
  - Cellは外部に見せない
  - リスト構造のリンクを直接触らせない

ライブラリ化の意義

- 利用側に、提供側のルールを守らせる
  - 具体的なリスト構造にアクセスできない
    ＝ リンクの構造を勝手に変更させない
- 実装方法を変更しても利用側は変更不要
  - 公開したメソッドを変えなければよい

➡情報隠蔽・カプセル化

補足：アクセス指定子

- public
  - 全てのクラス ＝ 全世界から参照可
- protected
  - 同じパッケージ内 ＝ 一緒に使うクラスから参照可
  - 継承したサブクラスからも参照可
- private
  - 同じクラスからだけ参照可 ＝ 他のクラスには内緒
  - 中身を隠しておくことで、より効率のよい実装に変更するなどが簡単になる
- デフォルト（何も書かない）
  - protectedから、サブクラスを除いたもの
  - パッケージプライベートともいう

レポート5 (1)

- ListLibパッケージを適切に変更し、リストによる実装を**配列による実装**に変更せよ
  - ただし、publicとして公開されたメソッドを変更してはならない ＝ ListLibMain側は変更してはならない
  - 詳細は次頁以降
- 提出期限：10/23(金) 12:55
  - 提出物はプロジェクト全体のjarファイル ← 後述
    - ソースとメインクラス指定を忘れずに
  - ファイル名は ListLibArray.jar とする

レポート5 (2)

- 配列を使うので、Cellクラスは不要になる
- 従来のListクラス
  - 変数headが、先頭のCellを指していた
    - headから順にnextをたどることで、全データにアクセスできた
- 今回のListクラス
  - headの代わりに int[] 型の変数 values を宣言して、適切な大きさのint配列をnewして代入する

レポート5 (3)

- 基本課題（必須）
  - insert(), append() を実装し直すこと
    - 配列のどの場所までデータを登録済みか ＝ 配列内のデータ末尾を記憶する必要があることに注意
    - insert() では、登録済みのデータを後ろにずらす必要があることに注意

レポート5 (4)

- 基本課題（つづき）
  - 明星LMSに雛形のList.javaがあるので、ダウンロードし、FIXMEと書かれた部分を埋めていく
    - パッケージ宣言は各自で
- 基本課題ではdelete()を実装しないので、delete()は動作しない ＝ コンソールの表示は削除前のもののまま

レポート5 (5)

- 発展課題1
  - 内部で用意した配列の大きさが不足した場合を想定すること

1. insert(), append()のときに、配列が満杯かチェック
2. 満杯 → 2倍の大きさの配列を作成
3. values に新しい配列を代入

レポート5 (6)

- 発展課題2
  - delete()も実装し直すこと
    1. 新しい配列を用意
    2. 削除対象以外の要素をコピー
    3. valuesに、新しい配列を代入
    4. 「配列内のデータ末尾」の印も更新

プログラムの配置(1)

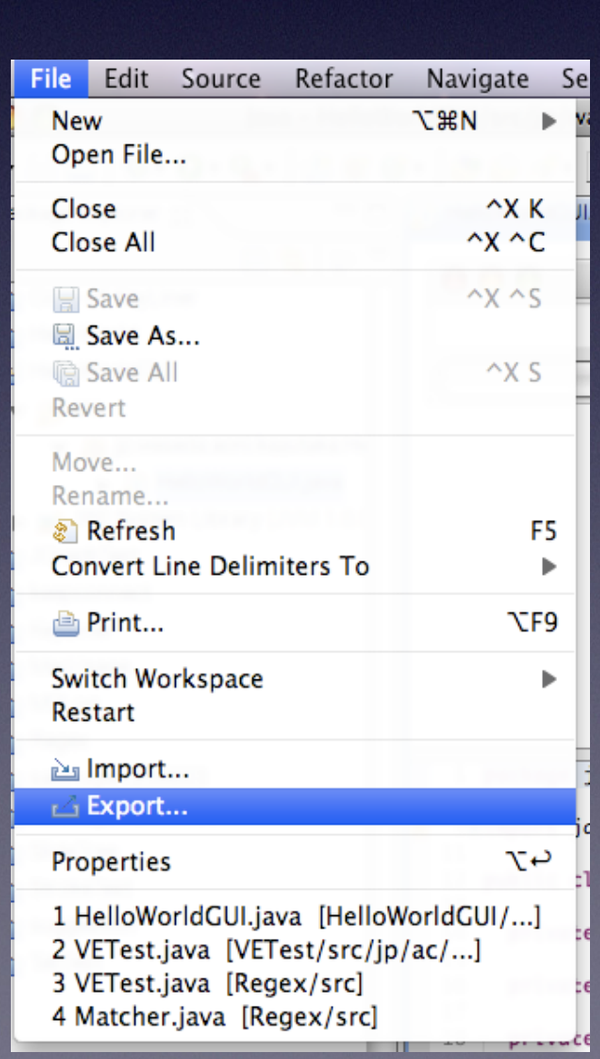
- 作成したプログラムは、最終的に実行できる状態で配置（デプロイ）する
  - 「実行ファイル作成」のようなもの
- クラス数が少なければ…
  1. 全クラスファイルを同じ場所に置く
  2. main()のあるクラスをjavaコマンドで実行する

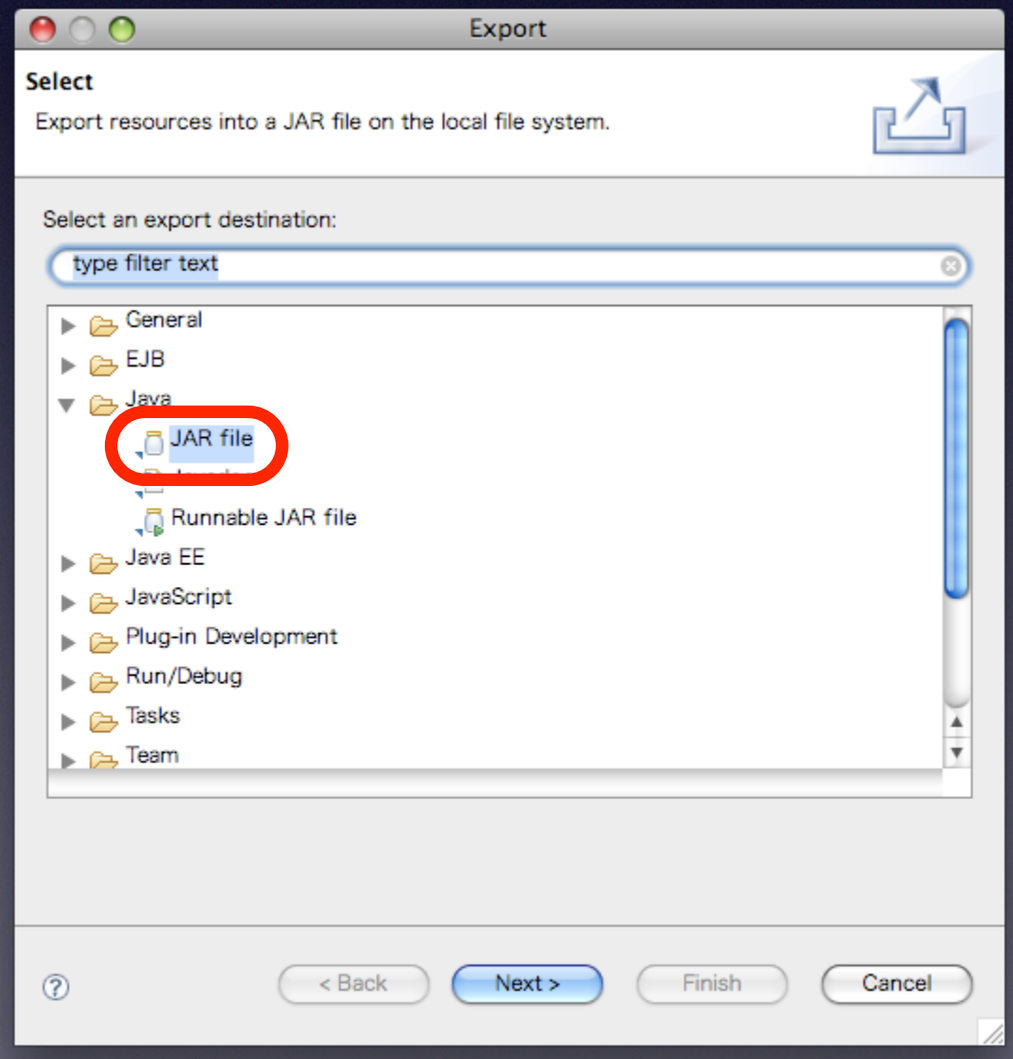
プログラムの配置(2)

- クラスファイルが増えると事実上不可能になる
  → jarファイルの利用
- jar(Java archive)ファイルとは…
  - 関連するクラスファイルと実行に必要な情報をzip形式で固めたもの
  - 作成したプログラム（クラスファイル群）はこの形式にまとめるとよい

jar形式の書き出し(1)

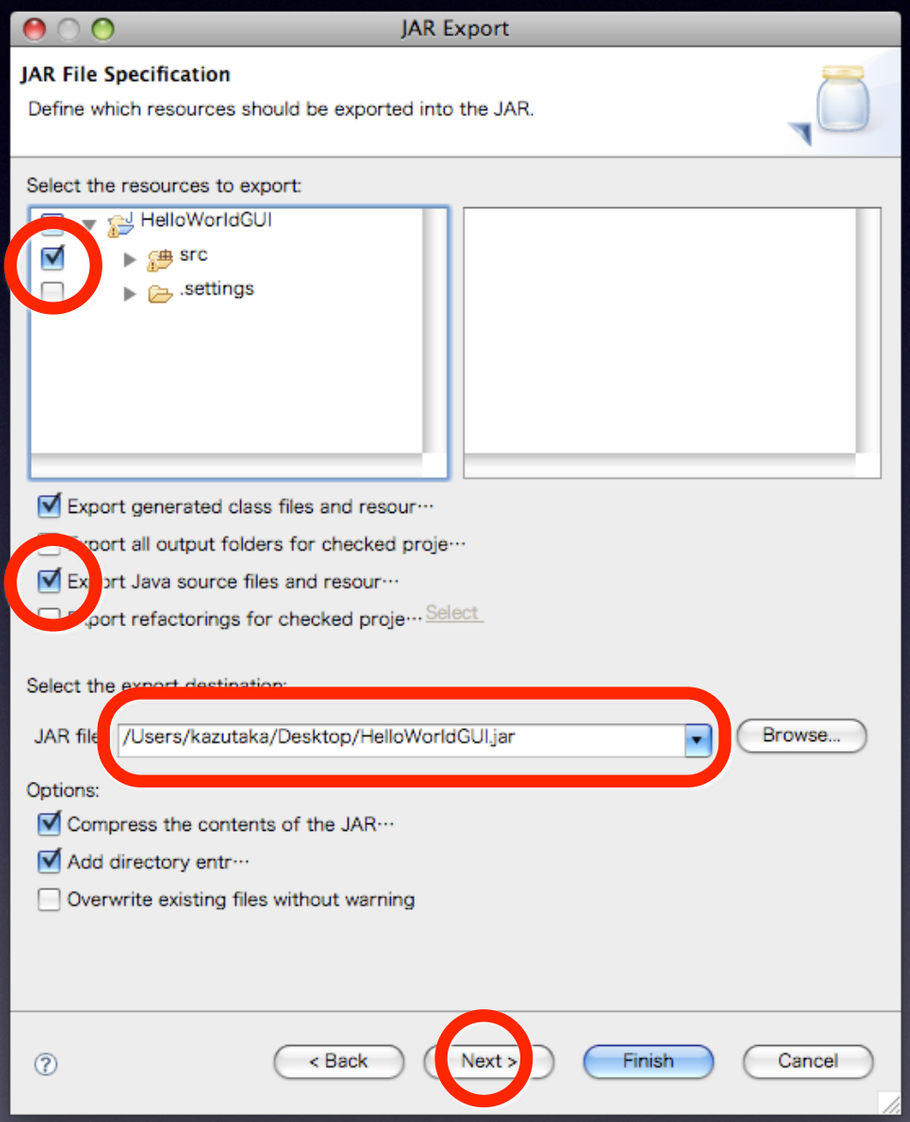
- File > Export
- Java > JAR fileを選択してNext





jar形式の書き出し(2)

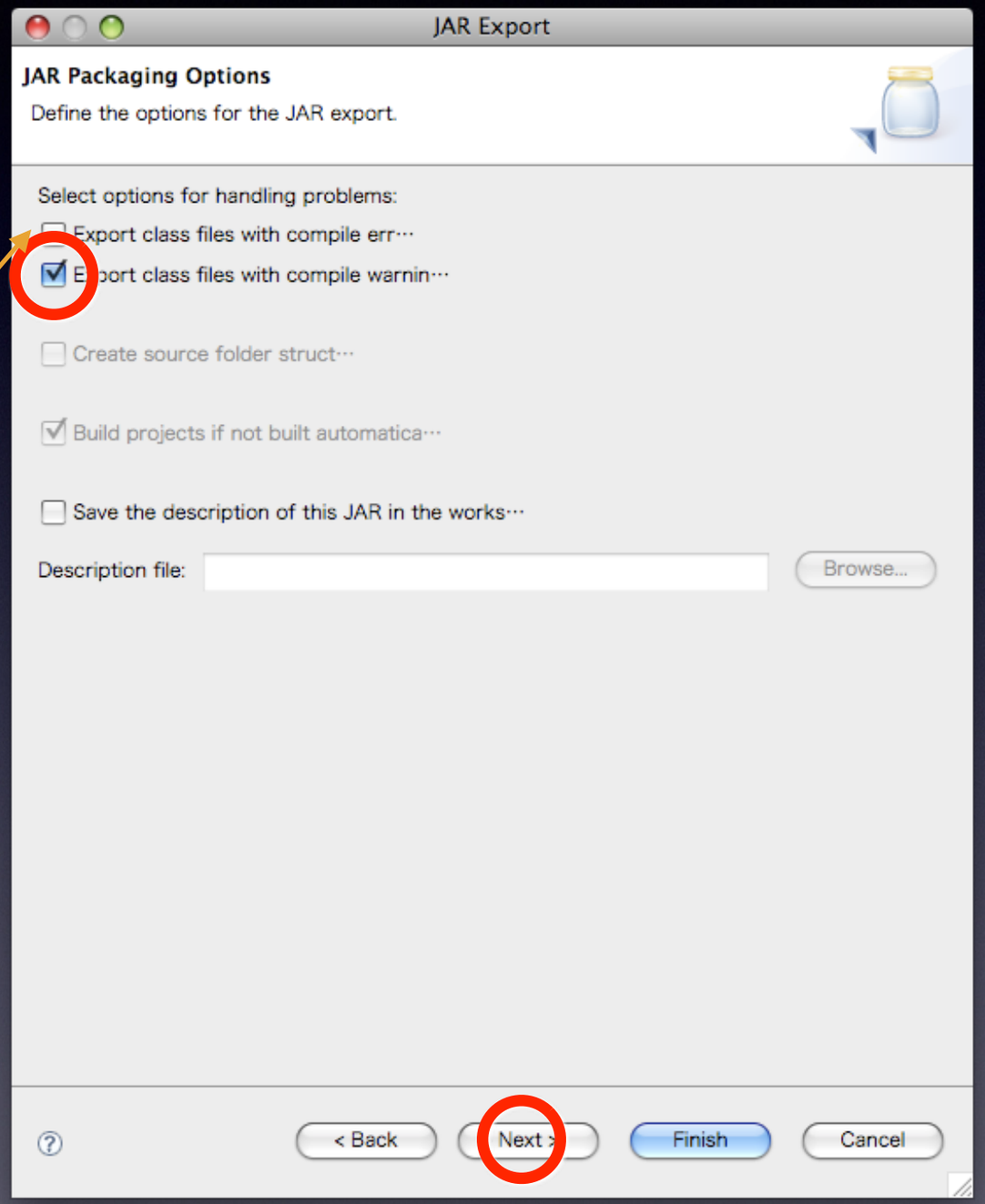
- ソースコードを含めるため、srcにチェックを入れる
- 出力するJAR fileの名前を書く
- 今回の課題は、デスクトップ上にListLibArray.jarで



（これはHelloWordGUIの例）

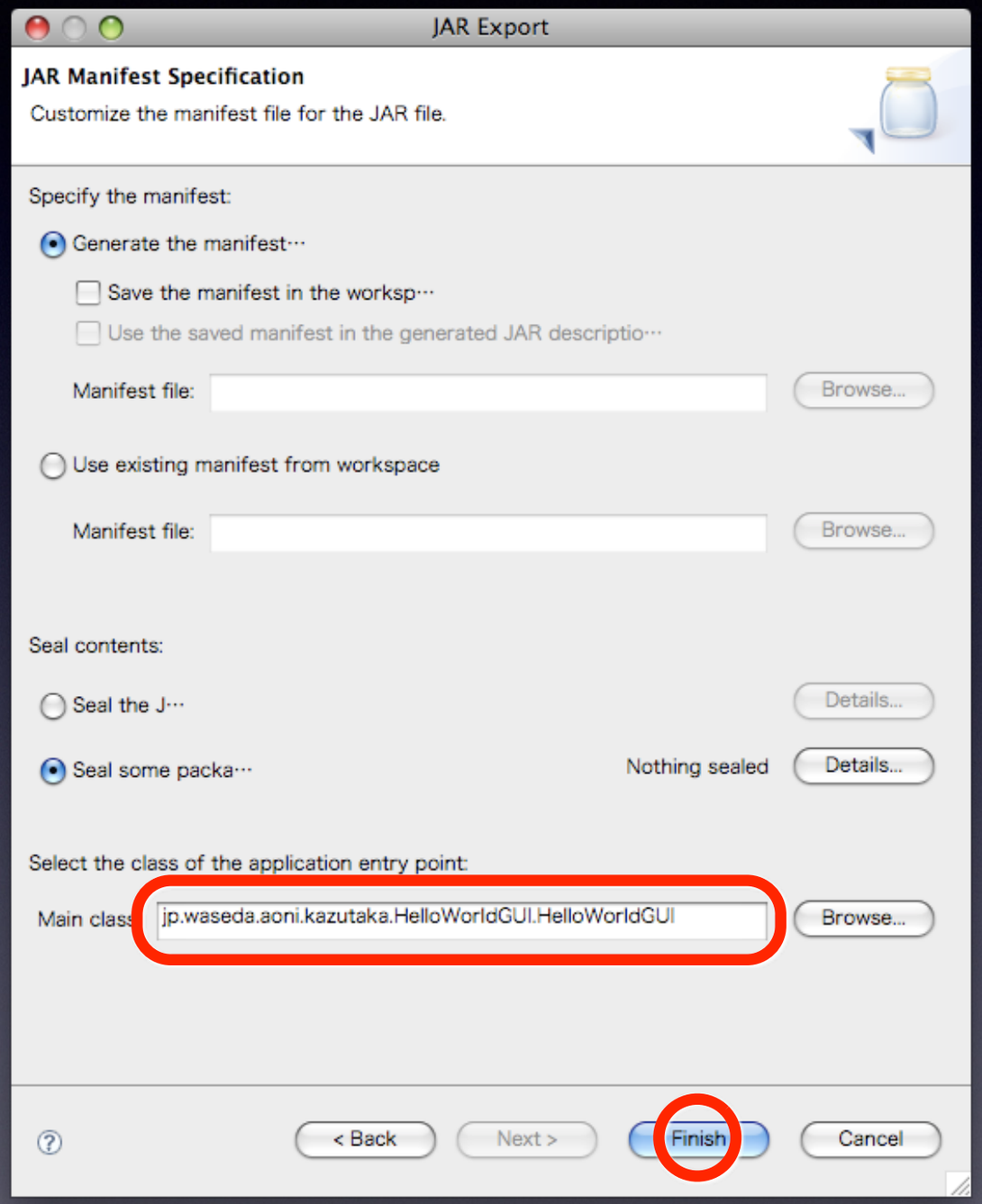
jar形式の書き出し(3)

- コンパイラのウォーニングがあってもJARに含めるよう、チェックを入れる
- 完成せず、エラーがあるファイルを提出する場合はその上もチェック



jar形式の書き出し(4)

- main()メソッドのあるクラスを選択しておく
- 今回の課題ではListLibSample
- JARファイルのダブルクリックで実行できる



jar形式の書き出し(5)

- 仮想Windows環境でjarファイルを実行するには
  - 端末でjarファイルの場所に移動+コマンド実行
  - 端末 = コマンドプロンプト(Win)、ターミナル(Mac)、GNOME端末など(Linux)

```
C:\Windows\system32>cd \Users\自分のユーザ名\Desktop
C:\Users\自分のユーザ名\Desktop>dir
 (表示された一覧に、エクスポートしたjarファイルがあるはず)
C:\Users\自分のユーザ名\Desktop>java -jar
ListLibArray.jar
```

(\ は円マークで表示されるかも)

jar形式の書き出し(6)

- JARファイルの実態はzipファイルなので、拡張子を変更すれば展開することも可能
- 端末でファイルの内容を確認するには、jar コマンドを用いる

```
C:¥Windows¥System32¥cmd.exe
Microsoft Windows [Version 10.0.19044.6093]
(c) Microsoft Corporation. All rights reserved.

C:¥Windows¥system32>cd ¥Users¥j212226w¥Desktop

C:¥Users¥j212226w¥Desktop>java -jar ListLibArray.jar
3 0

C:¥Users¥j212226w¥Desktop>jar -tf ListLibArray.jar
META-INF/MANIFEST.MF
.project
.classpath
jp/ac/meiseiu/maruyama/kazutaka/ListLib/List.class
jp/ac/meiseiu/maruyama/kazutaka/ListLib/List.java
jp/ac/meiseiu/maruyama/kazutaka/ListLibSample/ListLibSample.class
jp/ac/meiseiu/maruyama/kazutaka/ListLibSample/ListLibSample.java
```

ソースコードが含まれている

パッケージ階層 ＝ ディレクトリ階層

以上はレポート提出用

- レポート提出時はJavaソースファイルも含める必要があるので、前頁までの手順で実行する
- 一般に「完成版のプログラム」を出荷する場合にソースは含めないのでRunnable JARを用いるが、レポート提出で使うのはNGなので説明は省略